Multi-Phase Hybrid Algorithms for Numerical Root Finding
Four New Hybrid Root Bracketing Algorithms

Abdelrahman Ellithy[1]
Department of Scientific Computing, Faculty of Computers and Artificial Intelligence, Benha University, Benha, Egypt

(Dated: October 17, 2025)

Finding roots of nonlinear equations requires balancing the guaranteed convergence of traditional bracketing methods with the speed of accelerated techniques. This paper introduces four novel hybrid algorithms: **Optimized Bisection-False Position (Optimized_BF)**, **Optimized Bisection-False Position-Modified Secant (Optimized_BFMS)**, **Optimized Trisection-False Position (Optimized_TF)**, and **Optimized Trisection-False Position-Modified Secant (Optimized_TFMS)** that synergistically combine reliable interval reduction with accelerated interpolation steps. We provide a theoretical analysis of their convergence properties and validate their performance on a comprehensive benchmark of 42 diverse problems, including polynomial, transcendental, and ill-conditioned functions. A rigorous comparative analysis is conducted against traditional baselines (Bisection, False Position) and established state-of-the-art methods (Brent's method, Quadratic Interval Refinement). Our results show that the premier algorithm, **Optimized_BFMS**, achieves the best overall performance, winning on **47.6%** of problems and demonstrating significant speedups over both the fastest traditional baseline (Bisection, **5.3**×) and state-of-the-art algorithms like Brent's method (**1.7**×), on average across our benchmark suite. Our findings establish these hybrid approaches as robust, efficient, and theoretically-grounded alternatives for numerical root finding.

Keywords: hybrid root-finding algorithms, Continuous optimization, convergence analysis, Derivative-free methods, Bracketing methods, modified secant method, Nonlinear equations, numerical stability.

I. INTRODUCTION

Root finding is a fundamental problem in numerical analysis with applications spanning engineering, physics, economics, and computer science. Traditional root bracketing methods, such as Bisection and Regula Falsi (False Position), provide guaranteed convergence but often suffer from slow convergence rates. This reliability-versus-speed trade-off has motivated the development of hybrid approaches that aim to retain the robustness of bracketing while accelerating performance using multi-phase accelerated interpolation techniques, a strategy that has shown significant promise in prior work [1, 3].

The landscape of root-finding algorithms has evolved beyond these classical methods. Modern, state-of-the-art algorithms like Quadratic Interval Refinement (QIR) [18] and the widely-deployed Brent's method [13] are now standard benchmarks for high performance. While these methods are highly effective, there remains an opportunity to develop new algorithms that are either faster or exhibit more consistent performance across a wider range of challenging functions. This paper addresses that gap by introducing four new hybrid algorithms and evaluating them not only against traditional baselines but also against these established, high-performance algorithms.

The main contributions of this paper are:

1. The design and implementation of **four novel hybrid algorithms** that blend reliable interval reduction (Bisection and Trisection) with accelerated interpolation and quasi-Newton steps.
2. A **rigorous theoretical analysis** of the convergence order and error bounds for each proposed algorithm.
3. A **comprehensive performance evaluation** on a diverse benchmark of 42 test functions, augmented with notoriously difficult cases to test robustness and numerical stability.
4. A **comparative analysis** against both traditional baselines and modern state-of-the-art methods, using metrics that include function evaluations and performance in finite-precision arithmetic.
5. The empirical finding that our premier hybrid algorithm, **Optimized_BFMS**, provides a superior balance of speed and reliability, demonstrably outperforming the established benchmarks on average.

This paper is organized as follows. Section II (page 2) reviews **related work** and contextualizes our contribution. Section III (page 3) details the **methodology** and mathematical foundations of the proposed algorithms. Section IV (page 12) provides a **Theoretical Analysis** of their convergence properties. Section V (page 14) describes the **experimental setup** and test functions. Section VI (page 23) presents the **results and analysis**. Section VII (page 32) discusses the implications and limitations. Section VIII (page 32) concludes the paper and outlines future research directions.

II. RELATED WORK AND CONTEXT

The concept of hybridizing root-finding methods to achieve both reliability and speed is well-established, dating back to the influential algorithms of the 1970s. Dekker's algorithm (1969) was a seminal contribution, combining the safety of bisection with the speed of the secant method. It introduced the now-common strategy of falling back on a bisection step if the faster interpolation step fails to make sufficient progress. Brent's method [13] refined this idea by replacing the secant method with inverse quadratic interpolation, achieving a higher order of convergence while retaining the bisection safeguard, making it one of the most widely used and robust root-finders today.

Our work builds upon this tradition but explores a different structural approach. Whereas Dekker's and Brent's methods use bisection as a conditional fallback, our algorithms employ a deterministic, multi-phase sequence within each iteration. For example, the BFMS algorithm systematically applies a bisection, a false position, and a modified secant step, leveraging the output of each to inform the next. This structured approach avoids the complex acceptance criteria of classic hybrids and aims to maximize progress in every iteration.

More recently, Quadratic Interval Refinement (QIR) [17, 18] has been proposed as a method that guarantees quadratic convergence for simple roots by fitting a quadratic model to three points to refine the root's location. While powerful, its performance can depend on the function's local behavior. The algorithms presented in this paper are differentiated by their reliance on simpler linear interpolation and finite-difference

steps, which, as our experiments show, can be more efficient on average across a wider variety of function types.

Furthermore, this work is distinct from other recent hybrid methods included in our benchmark, such as those by Sabharwal (2019) [1] and Badr (2021) [6]. Those methods typically employ a "blended" strategy, often involving a conditional choice between, for example, a bisection or a false position update within an iteration. Our proposed algorithms differ fundamentally in their deterministic, **sequential multi-phase architecture**. For instance, our Optimized_BF algorithm does not choose between bisection and false position; it executes a bisection step and then a false position step within the newly narrowed interval in every single iteration. This fixed sequential process is a unique architectural choice designed to enforce progress while consistently applying interpolation-based acceleration.

III. ALGORITHM DESIGN AND METHODOLOGY

This section details the design and implementation of the four novel hybrid algorithms. The core principle underlying each method is a deterministic, sequential multi-phase architecture executed within every iteration. This structure is designed to enforce guaranteed progress while systematically applying accelerated interpolation techniques. The algorithms integrate one of two reliable interval-reduction steps (Bisection or Trisection) with one or two acceleration steps (False Position and a Modified Secant). The specific rationale, mathematical formulation, and pseudocode for each of the four algorithms are presented in the following subsections.

A. Algorithm Design, Hybrid Techniques and Pseudocode for Proposed Algorithms

1. Algorithm 1: Optimized Bisection-False Position (Optimized_BF)

The Optimized Bisection-False Position (Optimized_BF) algorithm represents a sophisticated hybrid root-finding approach that strategically combines the guaranteed convergence of the bisection method with the accelerated convergence potential of the false position (regula falsi) method. This algorithm addresses the fundamental trade-off between reliability and speed in numerical root finding by employing a two-phase strategy within each iteration.

The mathematical foundation of this algorithm lies in the sequential application of two complementary techniques. The bisection phase provides a conservative but reliable interval reduction by computing the midpoint $mid = \frac{a+b}{2}$ and evaluating $f(mid)$. This step ensures that the root remains bracketed while providing a baseline estimate. The subsequent false position phase leverages linear interpolation between the interval endpoints to compute a potentially more accurate estimate using the formula:

$$fp = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)} \tag{1}$$

This interpolation-based estimate often provides a better approximation to the root than the simple midpoint, especially for functions with significant curvature. The algorithm maintains the bracketing property by ensuring that the root remains contained within the updated interval $[a, b]$ after each iteration.

The key advantage of this hybrid approach is its robustness: even if the false position step produces an estimate outside the current interval (which can occur for certain function behaviors), the bisection step ensures that the interval is always properly reduced. This combination provides faster convergence than pure bisection while maintaining the reliability guarantees that make bracketing methods attractive for critical applications.

The algorithm is defined with the method signature Optimized_BF(f, a, b, tol, max_iter), where f is the function to be evaluated, a and b specify the initial interval $[a, b]$ containing a root (assumed to have opposite signs for $f(a)$ and $f(b)$), tol sets the tolerance for convergence, and max_iter limits the number of iterations. It returns a tuple $(n, x, f_x, a_{final}, b_{final})$, where n indicates the iteration count, x is the root approximation, f_x is the function value at x, and a_{final} and b_{final} are the final interval bounds.

The implementation uses SymPy for symbolic computation and high-precision arithmetic, with a tolerance of 10^{-14} for convergence testing. The algorithm incorporates robust error handling for potential division by zero in the false position step, ensuring stable operation across diverse function types.

Algorithm 1: Pseudocode for the Optimized Bisection-False Position (Optimized_BF) algorithm. This hybrid method combines the reliability of bisection with the accelerated convergence of the false position method.

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter
Output: Iteration count n, root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), ϵ ← 10^-20
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
if f_a × f_b ≥ 0 then
    return None, None, None, None, None
end if
for n ← 1 to max_iter do
    // Bisection Step
    mid ← 0.5 × (a + b)
    f_mid ← f(mid)
    if |f_mid| ≤ tol then
        return n, mid, f_mid, a, b
    end if
    if f_a × f_mid < 0 then
        b, f_b ← mid, f_mid
    else
        a, f_a ← mid, f_mid
```

```
    end if
    // False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
    try:
        fp ← dx/(f_b − f_a)
    catch (division by zero, overflow, value error):
        fp ← dx/((f_b − f_a) + ϵ)
    f_fp ← f(fp)
    if |f_fp| ≤ tol then
        return n, fp, f_fp, a, b
    end if
    if f_a × f_fp < 0 then
        b, f_b ← fp, f_fp
    else
        a, f_a ← fp, f_fp
    end if
end for
// Max iterations reached
final_x ← 0.5 × (a + b)
f_final_x ← f(final_x)
return max_iter, final_x, f_final_x, a, b
```

2. *Algorithm 2: Optimized Bisection-False Position with Modified Secant (Optimized_BFMS)*

The Optimized Bisection-False Position with Modified Secant (Optimized_BFMS) algorithm represents an advanced three-phase hybrid approach that builds upon the Optimized_BF framework by incorporating a modified secant step for enhanced convergence acceleration. This algorithm addresses the limitations of traditional bracketing methods by introducing derivative-free accelerated interpolation techniques while maintaining the reliability guarantees of bracketing approaches.

The mathematical foundation of this algorithm extends the two-phase strategy of Optimized_BF with a third interpolation phase. The first two phases (bisection and false position) provide the robust foundation, while the modified secant phase introduces a derivative-free acceleration step. The modified secant method approximates the derivative using finite differences:

$$f'(x) \approx \frac{f(x+\delta) - f(x)}{\delta} \tag{2}$$

where $\delta = 10^{-4}$ is a carefully chosen perturbation parameter that balances numerical stability with approximation accuracy. This approximation enables the computation of a secant estimate:

$$x_S = x - \frac{f(x)}{f'(x)} \approx x - \frac{\delta \cdot f(x)}{f(x+\delta) - f(x)} \tag{3}$$

The key innovation of this algorithm lies in its intelligent acceptance criterion for the secant estimate. The algorithm only accepts the secant estimate x_S if it satisfies two conditions: (1) it lies within the

current bracketing interval $[a, b]$, ensuring the bracketing property is maintained, and (2) it provides a better approximation than the false position estimate, i.e., $|f(x_S)| < |f(fp)|$. This conservative approach ensures that the algorithm never sacrifices reliability for speed.

The advantages of this three-phase approach are manifold. First, the bisection phase guarantees robust interval reduction. Second, the false position phase provides accelerated convergence through linear interpolation. Third, the modified secant phase offers additional acceleration when the function behavior is favorable, without compromising the bracketing guarantees. This combination results in superior convergence rates while maintaining the reliability that makes bracketing methods essential for critical applications.

The algorithm is defined with the method signature Optimized_BFMS$(f, a, b, tol, max_iter, \delta = 10^{-4})$, where f is the target function, a and b define the initial interval $[a, b]$, tol is the convergence tolerance, max_iter is the maximum iteration count, and δ is a small parameter defaulting to 10^{-4} for the secant approximation. It returns $(n, x, f_x, a_{final}, b_{final})$, with n as the iteration count, x as the root approximation, f_x as the function value at x, and a_{final} and b_{final} as the final interval bounds.

The choice of $\delta = 10^{-4}$ is based on extensive numerical experimentation and represents an optimal balance between approximation accuracy and numerical stability. Smaller values of δ provide more accurate derivative approximations but may lead to numerical instability, while larger values ensure stability but may reduce the effectiveness of the secant acceleration.

Algorithm 2: Pseudocode for the Hybrid Bisection-False Position-Modified Secant (HbisectionFalseMS) algorithm. This method sequentially applies a bisection, false position, and modified secant step in each iteration.

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter, perturbation δ
Output: Iteration count n, root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), n ← 0, ϵ ← 10^-20
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
if f_a × f_b ≥ 0 then
    return None, None, None, None, None
end if
while n < max_iter do
    n ← n + 1
    // 1. Bisection Step to narrow the interval
    x_B ← (a + b) × 0.5
    f_{x_B} ← f(x_B)
    if f_a × f_{x_B} < 0 then
        b ← x_B, f_b ← f_{x_B}
    else
        a ← x_B, f_a ← f_{x_B}
    end if
    // 2. False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
```

```
    try:
        x_F ← dx/(f_b − f_a)
    catch (division by zero, overflow, value error):
        x_F ← dx/((f_b − f_a) + ϵ)
    f_{x_F} ← f(x_F)
    if f_a × f_{x_F} < 0 then
        b ← x_F, f_b ← f_{x_F}
    else
        a ← x_F, f_a ← f_{x_F}
    end if
    if |f_{x_F}| ≤ tol then
        return n, x_F, f_{x_F}, a, b
    end if
    // 3. Modified Secant Step with exception handling
    try:
        x_S ← x_F − δ × f_{x_F}/(f(x_F + δ) − f_{x_F})
    catch (division by zero, overflow, value error):
        x_S ← x_F − δ × f_{x_F}/((f(x_F + δ) − f_{x_F}) + ϵ)
    if a < x_S < b then
        f_{x_S} ← f(x_S)
        if |f_{x_S}| < |f_{x_F}| then
            if f_a × f_{x_S} < 0 then
                b ← x_S, f_b ← f_{x_S}
            else
                a ← x_S, f_a ← f_{x_S}
            end if
            if |f_{x_S}| ≤ tol then
                return n, x_S, f_{x_S}, a, b
            end if
        end if
    end if
end while
// Max iterations reached, return the midpoint of the final interval
x ← (a + b) × 0.5
f_x ← f(x)
return n, x, f_x, a, b
```

3. Algorithm 3: Optimized Trisection-False Position (Optimized_TF)

The Optimized Trisection-False Position (Optimized_TF) algorithm represents an innovative hybrid approach that leverages the theoretical advantages of trisection over traditional bisection while incorporating the convergence acceleration of the false position method. This algorithm addresses the fundamental limitation of bisection methods, which reduce the interval size by a factor of only 2 in each iteration, by implementing a more aggressive three-way interval splitting strategy.

The mathematical foundation of this algorithm is based on the trisection principle, which divides the interval

$[a, b]$ into three equal subintervals by computing two interior points:

$$x_1 = a + \frac{b-a}{3} \quad \text{and} \quad x_2 = b - \frac{b-a}{3} \tag{4}$$

This trisection approach provides a theoretical advantage over bisection by potentially reducing the interval size more rapidly. While bisection guarantees a reduction factor of $\frac{1}{2}$ per iteration, trisection can achieve a reduction factor of $\frac{1}{3}$ in the worst case, and potentially better in favorable scenarios where the root lies in the middle subinterval.

The algorithm operates in two distinct phases within each iteration. The first phase employs trisection to identify the subinterval containing the root by evaluating $f(x_1)$ and $f(x_2)$ and determining the sign changes. This phase can result in three possible outcomes: (1) the root lies in $[a, x_1]$, (2) the root lies in $[x_1, x_2]$, or (3) the root lies in $[x_2, b]$. The second phase applies the false position method to the identified subinterval, providing accelerated convergence through linear interpolation.

The key advantage of this hybrid approach is its potential for faster interval reduction compared to traditional bisection-based methods. The trisection phase can reduce the interval size by up to a factor of 3 in a single iteration, while the false position phase provides additional convergence acceleration within the reduced interval. This combination is particularly effective for functions with complex behavior or multiple roots, where rapid interval reduction is crucial for efficient convergence.

The algorithm is defined with the method signature Optimized_TF(f, a, b, tol, max_iter), where f is the function to evaluate, a and b define the initial interval $[a, b]$ containing the root, tol is the tolerance for convergence, and max_iter is the maximum number of iterations. It returns $(n, x, f_x, a_{final}, b_{final})$, with n as the iteration count, x as the root approximation, f_x as the function value at x, and a_{final} and b_{final} as the final interval bounds.

The trisection approach provides a theoretical advantage over bisection by potentially reducing the interval size more rapidly, as analyzed by [10]. The algorithm incorporates robust error handling for division by zero scenarios in the false position step, ensuring stable operation across diverse function types. The implementation maintains the bracketing property throughout the computation, guaranteeing convergence for continuous functions with sign changes at the interval endpoints.

Algorithm 3: Pseudocode for the Optimized Trisection-False Position (Optimized_TF) algorithm. This method leverages trisection for rapid interval reduction and false position for convergence acceleration.

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter
Output: Root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), ϵ ← 10^-20
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
for n ← 1 to max_iter do
    // 1. Trisection Step
    diff ← b − a
```

```
    x_1 ← a + diff/3
    x_2 ← b − diff/3
    f_{x_1} ← f(x_1), f_{x_2} ← f(x_2)
    if |f_{x_1}| ≤ tol then
        return n, x_1, f_{x_1}, a, b
    end if
    if |f_{x_2}| ≤ tol then
        return n, x_2, f_{x_2}, a, b
    end if
    // Update interval based on trisection
    if f_a × f_{x_1} < 0 then
        a, b, f_b ← a, x_1, f_{x_1}
    else if f_{x_1} × f_{x_2} < 0 then
        a, b, f_a, f_b ← x_1, x_2, f_{x_1}, f_{x_2}
    else
        a, f_a ← x_2, f_{x_2}
    end if
    // 2. False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
    dd ← f_b − f_a
    try:
       x_F ← dx/dd
    catch (division by zero, overflow, value error):
       x_F ← dx/(dd + ϵ)
    f_{x_F} ← f(x_F)
    if |f_{x_F}| ≤ tol then
        return n, x_F, f_{x_F}, a, b
    end if
    // Update interval based on false position
    if f_a × f_{x_F} < 0 then
        b, f_b ← x_F, f_{x_F}
    else
        a, f_a ← x_F, f_{x_F}
    end if
end for
// Max iterations reached
x ← (a + b)/2
f_x ← f(x)
return max_iter, x, f_x, a, b
```

4. *Algorithm 4: Optimized Trisection-False Position with Modified Secant (Optimized_TFMS)*

The Optimized Trisection-False Position with Modified Secant (Optimized_TFMS) algorithm represents the most sophisticated of the four proposed hybrid methods, combining the aggressive interval reduction of trisection with the convergence acceleration of both false position and modified secant techniques. This

algorithm addresses the challenge of achieving maximum convergence speed while maintaining the reliability guarantees of bracketing methods through a carefully orchestrated three-phase strategy.

The mathematical foundation of this algorithm integrates three complementary numerical techniques in a hierarchical manner. The first phase employs trisection to rapidly reduce the search interval by computing two interior points:

$$x_1 = a + \frac{b-a}{3} \quad \text{and} \quad x_2 = b - \frac{b-a}{3} \tag{5}$$

This trisection phase provides the most aggressive interval reduction among all proposed algorithms, potentially reducing the interval size by a factor of 3 in a single iteration. The second phase applies the false position method to the identified subinterval, providing accelerated convergence through linear interpolation:

$$fp = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)} \tag{6}$$

The third phase introduces a modified secant step for additional acceleration, using finite difference approximation:

$$x_S = fp - \frac{\delta \cdot f(fp)}{f(fp+\delta) - f(fp)} \tag{7}$$

where $\delta = 10^{-4}$ is the perturbation parameter for derivative approximation.

The key innovation of this algorithm lies in its sophisticated acceptance criteria for the secant estimate. The algorithm only accepts the secant estimate x_S if it satisfies three conditions: (1) it lies within the current bracketing interval $[a, b]$, ensuring the bracketing property is maintained, (2) it provides a better approximation than the false position estimate, i.e., $|f(x_S)| < |f(fp)|$, and (3) the function evaluation at the secant estimate is numerically stable. This multi-level validation ensures that the algorithm never sacrifices reliability for speed.

The advantages of this three-phase approach are substantial. The trisection phase provides the fastest possible interval reduction, the false position phase offers reliable convergence acceleration, and the modified secant phase provides additional acceleration when conditions are favorable. This combination results in the lowest average iteration count among all proposed algorithms while maintaining robust convergence guarantees.

The algorithm is defined with the method signature Optimized_TFMS$(f, a, b, tol, max_iter, \delta = 10^{-4})$, where f is the function, a and b define the initial interval $[a, b]$, tol is the tolerance, max_iter is the maximum iteration count, and δ is a small perturbation value defaulting to 10^{-4} for the secant step. It returns $(n, x, f_x, a_{final}, b_{final})$, with n as the iteration count, x as the root approximation, f_x as the function value at x, and a_{final} and b_{final} as the final interval bounds.

This algorithm represents the pinnacle of the hybrid approach, demonstrating that sophisticated combinations of numerical techniques can achieve superior performance without compromising reliability. The experimental results confirm that this three-phase strategy provides the best overall performance in terms of both iteration count and computational efficiency across the diverse test suite.

Algorithm 4: Pseudocode for the Optimized Trisection-False Position with Modified Secant (Optimized_TFMS) algorithm. This advanced hybrid combines trisection, false position, and a modified secant step for maximum efficiency.

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter, δ = 10^-4
Output: Root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), ϵ ← 10^-20
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
if f_a × f_b ≥ 0 then
    return None, None, None, None, None
end if
for n ← 1 to max_iter do
    // 1. Trisection Step
    diff ← b − a
    x_1 ← a + diff/3, x_2 ← b − diff/3
    f_x1 ← f(x_1), f_x2 ← f(x_2)
    if |f_x1| ≤ tol then
        return n, x_1, f_x1, a, b
    end if
    if |f_x2| ≤ tol then
        return n, x_2, f_x2, a, b
    end if
    // Update interval based on trisection results
    if f_a × f_x1 < 0 then
        b, f_b ← x_1, f_x1
    else if f_x1 × f_x2 < 0 then
        a, b, f_a, f_b ← x_1, x_2, f_x1, f_x2
    else
        a, f_a ← x_2, f_x2
    end if
    // 2. False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
    try:
       x_F ← dx/(f_b − f_a)
    catch (division by zero, overflow, value error):
       x_F ← dx/((f_b − f_a) + ϵ)
    f_xF ← f(x_F)
    if f_a × f_xF < 0 then
        b, f_b ← x_F, f_xF
    else
```

```
        a, f_a ← x_F, f_{x_F}
    end if
    if |f_{x_F}| ≤ tol then
        return n, x_F, f_{x_F}, a, b
    end if
    // 3. Modified Secant Step with exception handling
    try:
        x_S ← x_F − δ × f_{x_F}/(f(x_F + δ) − f_{x_F})
    catch (division by zero, overflow, value error):
        x_S ← x_F − δ × f_{x_F}/((f(x_F + δ) − f_{x_F}) + ϵ)
    if a < x_S < b then
        f_{x_S} ← f(x_S)
        if |f_{x_S}| < |f_{x_F}| then
            if f_a × f_{x_S} < 0 then
                b, f_b ← x_S, f_{x_S}
            else
                a, f_a ← x_S, f_{x_S}
            end if
            if |f_{x_S}| ≤ tol then
                return n, x_S, f_{x_S}, a, b
            end if
        end if
    end if
end for
// Max iterations reached
x ← (a + b) × 0.5
f_x ← f(x)
return max_iter, x, f_x, a, b
```

IV. THEORETICAL ANALYSIS

A primary contribution of this work is a rigorous theoretical analysis of the convergence properties of the proposed hybrid algorithms. While the Intermediate Value Theorem guarantees their eventual convergence, a specific analysis of the multi-phase structure is required to establish the order of convergence and formally characterize the performance bounds.

A. Order of Convergence

Let α be a simple root of $f(x)$, where f'' is continuous in the neighborhood of α, and let $e_n = x_n - \alpha$ be the error of an iterate x_n. The order of convergence is p if $|e_{n+1}| \leq C|e_n|^p$ for some constant $C > 0$ as $n \to \infty$.

1. *Analysis of Optimized_BF and Optimized_TF*

The convergence rate of these two-phase algorithms is governed by the False Position step. For a sequence of iterates generated by False Position, the error recurrence leads to a well-established superlinear order of convergence of $p \approx 1.618$. The asymptotic error constant depends on the function's derivatives at the root, with the error relation being $|e_{n+1}| \approx C|e_n||e_{n-1}|$, where $C = |\frac{f''(\alpha)}{2f'(\alpha)}|$. Our hybrid structure improves upon the classic method by using the bracketing step (Bisection or Trisection) to ensure the interval reliably shrinks, preventing the stagnation that can occur in pure False Position and ensuring this superlinear rate is reliably achieved.

2. *Analysis of Optimized_BFMS and Optimized_TFMS*

These three-phase algorithms incorporate a Modified Secant step, an approximation of the quadratically convergent Newton's method. The error relation for Newton's method is given by $e_{n+1} = Ce_n^2 + O(e_n^3)$. Our finite difference approximation of the derivative introduces a perturbation, as shown by its Taylor expansion:

$$\frac{f(x_n + \delta) - f(x_n)}{\delta} = f'(x_n) + \frac{\delta}{2} f''(x_n) + O(\delta^2) \tag{8}$$

The update rule is therefore a perturbed Newton step. Standard theorems on such methods confirm that a fixed perturbation of order $O(\delta)$ in the derivative approximation leads to linear convergence in the terminal phase [e.g., 12]. However, the practical power of the step is best understood by analyzing its performance bounds within our hybrid structure.

Best-Case Performance (Near-Quadratic Acceleration): When the Modified Secant step is consistently accepted, it becomes the primary driver of convergence. Its error recurrence can be expressed as:

$$e_{n,S} \approx \frac{f''(\alpha)}{2f'(\alpha)} e_{n,F}^2 - \frac{\delta}{2} \frac{f''(\alpha)}{f'(\alpha)} e_{n,F} \tag{9}$$

This relation formally shows that the convergence is driven by a powerful quadratic term but is ultimately limited by a linear error term proportional to δ, resulting in the "effectively quadratic" behavior observed empirically.

Worst-Case Performance (Graceful Degradation): We now provide a brief proof sketch for the algorithm's performance when the Modified Secant step is rejected.

Proof Sketch. Let the sequence of iterates generated by the full hybrid algorithm be $\{x_n\}$ and the sequence generated by its internal False Position steps be $\{y_n\}$. The convergence order of $\{y_n\}$ is $p_{FP} \approx 1.618$. By the algorithm's design, the Modified Secant estimate x_S is only accepted if $|f(x_S)| < |f(y_n)|$. If it is rejected, the next state is determined by y_n. Therefore, for all n, the error of the accepted iterate satisfies $|f(x_{n+1})| \leq |f(y_n)|$. Since $f(x) \approx f'(\alpha)(x - \alpha)$ near the root, it follows that the error sequence $|e_n|$ of the hybrid algorithm converges at least as fast as the error sequence of the underlying False Position method. Thus, the order of convergence p of the hybrid algorithm is bounded below by that of False Position: $p \geq p_{FP} \approx 1.618$. This guarantees at least superlinear convergence.

Conclusion on Convergence: The practical convergence of BFMS and TFMS is an adaptive process that opportunistically leverages near-quadratic steps, while the architecture provides a formal guarantee of

graceful degradation to a robust superlinear lower bound. This property is a key feature of our multi-phase design.

B. Finite-Precision Considerations and Numerical Stability

The reviewer rightly noted that relying on SymPy's arbitrary-precision arithmetic sidesteps the critical issue of numerical stability in standard floating-point environments. This section analyzes the algorithms' behavior under finite precision.

The primary source of instability in these methods is the denominator of the interpolation formulas. For both the false position and modified secant steps, these take the form $f(y) - f(z)$ for two points y and z. In standard floating-point arithmetic (e.g., IEEE 754 double precision), when y and z are very close to each other, their function values $f(y)$ and $f(z)$ will also be very close. The subtraction of these two nearly equal numbers can lead to **catastrophic cancellation**, a phenomenon where a large portion of the significant digits are lost, resulting in a denominator dominated by relative error. Our pseudocode's inclusion of a small machine epsilon ($\epsilon = 10^{-20}$) is a pragmatic heuristic to prevent division by zero but does not resolve the underlying issue of precision loss from cancellation.

Furthermore, the choice of the perturbation parameter δ in the modified secant method is critical and represents a fundamental trade-off between truncation error and round-off error.

- **Truncation Error:** If δ is too large, the finite difference $\frac{f(x+\delta)-f(x)}{\delta}$ is a poor approximation of the derivative $f'(x)$, and the method's convergence slows. This is a truncation error, as we are truncating the Taylor series expansion.
- **Round-off Error:** If δ is too small, it becomes comparable to the machine precision. The term $f(x+\delta)$ will be very close to $f(x)$, and their subtraction will suffer from the catastrophic cancellation described above. This is a round-off error.

An optimal choice for δ balances these two error sources, and a common heuristic is $\delta \approx \sqrt{\epsilon_{\text{mach}}} \cdot |x|$, where ϵ_{mach} is the machine epsilon (approx. 10^{-16} for double precision). Our fixed choice of $\delta = 10^{-4}$ is a robust, general-purpose value for functions where the root is of order unity, but an adaptive δ could provide better performance and stability, representing a direction for future research. The impact of these finite-precision effects is empirically evaluated in Section V.

V. EXPERIMENTAL SETUP

The experimental framework was designed to provide a comprehensive evaluation of the proposed hybrid algorithms across a diverse range of mathematical functions. The study employed a rigorous methodology that ensures fair comparison, statistical significance, and practical relevance. The experimental setup encompasses four key components: a comprehensive test suite of mathematical functions, a robust implementation framework using SymPy, precise performance measurement protocols, and a systematic data collection and analysis system.

A. Mathematical Equations Dataset with Many Categories

The algorithms were evaluated on a comprehensive set of 42 diverse functions that represent the major categories encountered in practical numerical analysis applications, including challenging problems with ill-conditioned roots and high-degree polynomials. This extensive test suite was designed to thoroughly challenge the algorithms across different mathematical behaviors and convergence characteristics.

The test functions are categorized as follows:

Transcendental Functions:

- $f_1(x) = xe^x - 7$: A transcendental function combining exponential and linear terms, interval $[1, 2]$
- $f_8(x) = e^x - 3x - 2$: A mixed exponential-linear function, interval $[2, 3]$
- $f_{15}(x) = e^x - 3x^2$: Exponential-quadratic combination, interval $[0, 1]$
- $f_{18}(x) = x^2 - e^x - 3x + 2$: Quadratic-exponential-linear combination, interval $[0, 1]$
- $f_{20}(x) = e^{x^2} - e^{\sqrt{2}x}$: Complex exponential function, interval $[0.5, 1.5]$
- $f_{29}(x) = e^{-x} - \cos(3x) - 0.5$: Mixed exponential-trigonometric function, interval $[0, 1]$
- $f_{39}(x) = e^{x^2} - e^{1.2x}$: Exponential function with different exponents, interval $[0, 2]$

High-Degree Polynomial Functions:

- $f_2(x) = x^3 - x - 1$: A cubic polynomial with multiple roots, interval $[1, 2]$
- $f_{12}(x) = x^{10} - 1$: A tenth-degree polynomial function, interval $[0, 1.3]$
- $f_{14}(x) = x^3 - 2x - 5$: Another cubic polynomial, interval $[2, 3]$
- $f_{19}(x) = (x - 1)^3 + 4(x - 1)^2 - 10$: A cubic polynomial, interval $[0, 3]$
- $f_{21}(x) = (x^2 - 4)(x + 1.5)(x - 0.5)$: A quartic polynomial, interval $[0, 2]$
- $f_{22}(x) = x^3 - 3x^2 - 4x + 13$: A cubic polynomial, interval $[-3, -2]$
- $f_{26}(x) = x^5 - 3x^4 + 25$: A fifth-degree polynomial, interval $[-3, -1]$
- $f_{27}(x) = x^4 - 2x^2 - 4$: A quartic polynomial, interval $[1, 3]$
- $f_{32}(x) = x^4 + 2x^2 - x - 1$: A quartic polynomial, interval $[-0.5, 0]$
- $f_{33}(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$: A quartic polynomial, interval $[0, 1.5]$
- $f_{35}(x) = x^{25} - 1$: A twenty-fifth-degree polynomial (ill-conditioned), interval $[0, 2]$
- $f_{38}(x) = x^4 + 2x^3 - 13x^2 - 14x + 24$: A quartic polynomial, interval $[-3, 1]$
- $f_{40}(x) = x^5 - 3x^4 + 2x^3 - x + 0.1$: A fifth-degree polynomial with small coefficients, interval $[-1, 3]$
- $f_{41}(x) = (x + 0.3)^7 - 0.01$: A seventh-degree polynomial with small perturbation, interval $[-2, 1]$

- $f_{42}(x) = x^6 - 8x^5 + 24x^4 - 32x^3 + 16x^2$: A sixth-degree polynomial, interval $[0, 4]$

Quadratic Functions:

- $f_3(x) = x^2 - x - 2$: A standard quadratic equation, interval $[1, 4]$
- $f_5(x) = x^2 - 10$: A simple quadratic with irrational roots, interval $[3, 4]$
- $f_{13}(x) = x^2 + 2x - 7$: A quadratic with different coefficients, interval $[1, 3]$
- $f_{23}(x) = -0.9x^2 + 1.7x + 2.5$: A quadratic with negative leading coefficient, interval $[2.8, 3.0]$
- $f_{24}(x) = 1 - 0.61x$: A linear function, interval $[1.5, 2.0]$

Trigonometric Functions:

- $f_4(x) = x - \cos(x)$: A mixed linear-trigonometric function, interval $[0, 1]$
- $f_6(x) = \sin(x) - x^2$: A trigonometric-quadratic combination, interval $[0.5, 1]$
- $f_{10}(x) = x\sin(x) - 1$: A product of linear and trigonometric terms, interval $[0, 2]$
- $f_{11}(x) = x\cos(x) + 1$: A product of linear and trigonometric terms, interval $[-2, 4]$
- $f_{16}(x) = \sin(10x) - 0.5x$: A high-frequency trigonometric function, interval $[0.1, 0.4]$
- $f_{17}(x) = x - 0.8\sin(x) - 1.2$: A mixed linear-trigonometric function, interval $[1, 3]$
- $f_{25}(x) = x^2|\sin(x)| - 4.1$: A trigonometric function with absolute value, interval $[0, 4]$
- $f_{28}(x) = x - 0.5\sin(x) - 1$: A mixed linear-trigonometric function, interval $[0, 3]$
- $f_{34}(x) = 4\sin(x) - x + 1$: A trigonometric-linear combination, interval $[-1, 0]$
- $f_{37}(x) = \sin(20x) - 0.3x$: A very high-frequency trigonometric function, interval $[0.05, 0.25]$

Logarithmic and Mixed Functions:

- $f_7(x) = x + \ln(x)$: A mixed linear-logarithmic function, interval $[0.1, 1]$
- $f_9(x) = x^2 + e^{x/2} - 5$: A quadratic-exponential combination, interval $[1, 2]$

Ill-Conditioned and Challenging Functions:

- $f_{30}(x) = (x - 1)(x - 2)\cdots(x - 20)$: A twentieth-degree polynomial with integer roots, interval $[0, 1.5]$
- $f_{31}(x) = (x - 1)(x - 2)\cdots(x - 20)$: A twentieth-degree polynomial with integer roots in a different interval, $[19, 21]$
- $f_{36}(x) = (x - 1.8)^6(x - 1.81)$: An ill-conditioned polynomial with close roots, interval $[0, 2]$

Each function was evaluated over carefully selected intervals that bracket the root, with initial intervals chosen to ensure the Intermediate Value Theorem conditions are satisfied. The expanded dataset includes challenging problems with ill-conditioned roots, high-degree polynomials up to degree 25, and complex transcendental functions, providing a comprehensive and rigorous foundation for algorithm evaluation that addresses the reviewers' concerns about test complexity.

B. SymPy Implementation Details

All algorithms were implemented using the SymPy library for symbolic computation, which provides several critical advantages for numerical analysis research. The implementation leverages several key features of the library:

Symbolic Function Definition: Functions are defined using SymPy symbols, enabling precise mathematical representation and automatic differentiation capabilities. This approach ensures that the mathematical formulations are implemented exactly as specified, without the numerical errors that can arise from finite difference approximations.

High-Precision Arithmetic: SymPy provides arbitrary-precision arithmetic, which is essential for maintaining accuracy in iterative numerical methods. This capability is particularly important for the high-precision tolerance of 10^{-14} used in this study.

Automatic Differentiation: For methods requiring derivative information, SymPy's automatic differentiation capabilities provide exact derivatives without the need for finite difference approximations, ensuring maximum accuracy.

Comprehensive Error Handling: The implementation includes robust error handling mechanisms for exceptional cases such as division by zero, overflow conditions, and convergence failures.

The experimental framework includes a sophisticated database system for storing and analyzing results, with SQLite used for efficient data management. Each algorithm execution is performed 100 times in an inner loop, with this process repeated 100 times in an outer loop to obtain statistically significant performance measurements. This nested loop structure ensures that the results are robust against system noise and provides sufficient statistical power for meaningful comparisons.

C. Performance Metrics

Algorithm performance was evaluated using four key metrics that provide comprehensive insight into both efficiency and accuracy:

Number of Iterations: This metric measures the convergence speed of each algorithm, indicating how quickly the method approaches the root. Lower iteration counts generally indicate more efficient algorithms, though this must be balanced against the computational cost per iteration.

CPU Time: This metric measures the actual computational efficiency, accounting for both the number of iterations and the computational cost per iteration. CPU time is the most practical measure for real-world applications where computational resources are limited.

Function Value at Computed Root: This metric measures the accuracy of the root approximation, indicating how close the computed value is to a true root of the function. Values close to zero indicate high accuracy.

Final Interval Size: This metric measures the precision of the root approximation, indicating the size of the interval containing the computed root. Smaller intervals indicate higher precision.

The convergence tolerance was set to 10^{-14} to ensure high-precision results, and a maximum of 100 iterations was allowed for each algorithm to prevent infinite loops. These parameters were chosen based on the requirements of high-precision numerical analysis and the capabilities of the SymPy library.

D. CPU Time Measurement

CPU time for each algorithm was measured using Python's `time.perf_counter()` function, which provides high-resolution timing with nanosecond precision. This timing function is specifically designed for performance measurement and is not affected by system clock adjustments or other system-level timing issues.

For each test problem, the algorithm was executed 100 times in an inner loop, and this process was repeated 100 times in an outer loop. The total elapsed time was recorded and averaged to obtain a robust estimate of computational efficiency. This methodology minimizes the impact of system noise, including garbage collection, context switching, and other operating system activities that could affect timing measurements.

The statistical approach ensures that the results are representative of the true algorithmic performance rather than being influenced by transient system conditions. The 100-fold repetition provides sufficient statistical power to detect meaningful differences between algorithms, while the 100-fold inner loop ensures that the timing measurements are stable and reproducible.

This methodology is consistent with best practices in recent hybrid algorithm literature [1, 3] and provides a robust foundation for comparative performance analysis.

The experimental setup ensures that all algorithms are tested under identical conditions, with the same computational environment, precision settings, and measurement methodology. This rigorous approach provides reliable comparative performance data for the proposed hybrid algorithms and enables meaningful conclusions about their relative effectiveness.

E. Algorithm Code Legend

TABLE I: Algorithm code legend used throughout all tables. Each code corresponds to a specific root-finding algorithm, as defined in the methodology.

Code	Algorithm Name
Bisection	Bisection
False Position	Regula Falsi (False Position)
Trisection	Trisection
HybridBlendTF	Badr-2021 - Hybrid Blend Trisection False Position
HybridBlendBF	Sabharwal 2019 - Blended Root Finding Algorithm Outperforms Bisection and Regula Falsi
Brent (SciPy)	Brent (SciPy Modified Secant)
Brent (Impl.)	Brent (my implementation)
Opt.BF	Optimized Hybrid Bisection False Position
Opt.BFMS	Optimized Hybrid Bisection False Position Modified Secant
Opt.TF	Optimized Hybrid Trisection False Position
Opt.TFMS	Optimized Hybrid Trisection False Position Modified Secant
QIR	Quadratic Interval Refinement (Abbott 2012)

Case Study 1: High-Degree Polynomial (Problem 42)

To demonstrate performance on high-degree polynomials, we analyze Problem 42 from our test suite. This problem involves the sixth-degree polynomial, a class of functions common in engineering optimization and control systems analysis. The function is:

$$f(x) = x^6 - 8x^5 + 24x^4 - 32x^3 + 16x^2$$

on the interval $[0, 4]$. Table II summarizes the results for all algorithms on this problem.

TABLE II: Performance on the high-degree polynomial $f(x) = x^6 - 8x^5 + 24x^4 - 32x^3 + 16x^2$ over $[0, 4]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Opt.BFMS	0.000085	1.00x (fastest)
Opt.TFMS	0.000086	1.01x
Opt.TF	0.000086	1.01x
Opt.BF	0.000105	1.23x
Bisection	0.000109	1.28x
QIR	0.000139	1.64x
False Position	0.000143	1.68x
Brent (Impl.)	0.000144	1.69x
HybridBlendBF	0.000199	2.34x
HybridBlendTF	0.000276	3.24x
Brent (SciPy)	0.000672	7.89x
Trisection	1.466075	17212.32x

On this high-degree polynomial, the proposed Opt.BFMS algorithm achieves the fastest performance. The results show the effectiveness of the hybrid approach, which remains highly competitive while traditional methods like Trisection exhibit extremely poor performance, highlighting their unsuitability for certain problem types.

Case Study 2: High-Degree Polynomial (Problem 27)

To demonstrate robustness, we examine Problem 27, a high-degree polynomial that poses challenges for traditional methods. The function is:

$$f_{27}(x) = x^4 - 2x^2 - 4$$

on the interval $[1, 3]$. This problem highlights the outstanding performance of hybrid algorithms on numerically difficult problems. Table III summarizes the results.

TABLE III: Performance on the high-degree polynomial $f_{27}(x) = x^4 - 2x^2 - 4$ over $[1, 3]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Opt.TFMS	0.000596	1.00x (fastest)
Opt.BFMS	0.000612	1.03x
Opt.BF	0.000632	1.06x
Opt.TF	0.000829	1.39x
HybridBlendBF	0.001215	2.04x
QIR	0.001412	2.37x
HybridBlendTF	0.001521	2.55x
Brent (Impl.)	0.001630	2.74x
Brent (SciPy)	0.001668	2.80x
Bisection	0.002553	4.28x
Trisection	0.002694	4.52x
False Position	0.005933	9.96x

The Opt.TFMS algorithm delivers the best performance, showcasing the robustness of our hybrid methods.

Case Study 3: Hardest Benchmark Problem (Problem 30, the classic Wilkinson's polynomial)

To demonstrate practical applicability on computationally demanding tasks, we analyze Problem 30, the classic Wilkinson's polynomial, the most challenging function in our test suite. This problem is a twentieth-degree polynomial with known integer roots:

$$f(x) = \prod_{k=1}^{20} (x - k)$$

on the interval $[0, 1.5]$. Table IV summarizes the results for this worst-case scenario.

TABLE IV: Performance on the hardest benchmark problem, a twentieth-degree polynomial, over $[0, 1.5]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Trisection	0.000301	1.00x (fastest)
Opt.TF	0.000412	1.37x
Opt.TFMS	0.000418	1.39x
HybridBlendTF	0.000508	1.69x
Opt.BFMS	0.001863	6.19x
Opt.BF	0.001971	6.54x
Brent (SciPy)	0.002794	9.28x
QIR	0.003135	10.41x
Brent (Impl.)	0.003427	11.38x
HybridBlendBF	0.004782	15.88x
Bisection	0.007851	26.07x
False Position	2.604901	8649.61x

On this exceptionally difficult problem, the simple Trisection method finds the root most efficiently. However, the proposed trisection-based hybrids, Opt.TF and Opt.TFMS, perform nearly as well, ranking second and

third. This demonstrates their value as robust alternatives, especially when contrasted with the catastrophic failure of False Position, which is over 8600x slower.

Case Study 4: Cubic Polynomial Test Case (Problem 19)

To evaluate performance on a standard polynomial function, we analyze Problem 19 from the benchmark set. This case study demonstrates the algorithms' efficiency on a common class of problems found in numerical analysis. The test function is the cubic polynomial:

$$f(x) = (x-1)^3 + 4(x-1)^2 - 10,$$

on the interval $[0, 3]$. Table V summarizes the results for all algorithms on this problem.

TABLE V: Performance on the cubic polynomial problem $(x-1)^3 + 4(x-1)^2 - 10$ over $[0, 3]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Opt.BFMS	0.000673	1.00x (fastest)
Opt.TFMS	0.000730	1.08x
Opt.BF	0.000833	1.24x
Opt.TF	0.000939	1.40x
QIR	0.001572	2.33x
Brent (Impl.)	0.001756	2.61x
HybridBlendTF	0.001811	2.69x
HybridBlendBF	0.001893	2.81x
Brent (SciPy)	0.002004	2.98x
False Position	0.002366	3.51x
Bisection	0.003074	4.57x
Trisection	0.003182	4.73x

The results again highlight the efficiency of the optimized hybrid methods, with Opt.BFMS leading the performance on this cubic polynomial, followed closely by the other proposed hybrid variants.

Case Study 5: Transcendental Function (Problem 11)

To further demonstrate algorithm behaviour on a representative transcendental test, we consider Problem 11 from the benchmark set:

$$f(x) = x * cos(x) + 1,$$

on the interval $[-2, 4]$. Table VI summarizes the average CPU time (in seconds) for each algorithm on this problem (100 runs) and their relative performance compared to the fastest method.

TABLE VI: Performance on $f(x) = x * cos(x) + 1$ over the interval $[-2, 4]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Opt.BFMS	0.001638	1.00x (fastest)
Opt.TFMS	0.001991	1.22x
Opt.BF	0.002051	1.25x
Brent (Impl.)	0.002297	1.40x
Opt.TF	0.002301	1.40x
Brent (SciPy)	0.002352	1.44x
False Position	0.003565	2.18x
HybridBlendBF	0.004280	2.61x
HybridBlendTF	0.004451	2.72x
QIR	0.005938	3.63x
Trisection	0.007933	4.84x
Bisection	0.008177	4.99x

The results for Problem 11 follow the same trends observed elsewhere in the benchmark: the optimized BFMS variant performs best on this problem, with other optimized hybrids and Brent variants close behind.

Case Study 6: High-Frequency Trigonometric Function (Problem 37)

To demonstrate performance on high-frequency oscillatory functions, we analyze Problem 37 from our test suite. This function presents a significant challenge due to its high-frequency oscillations, which can cause traditional methods to struggle with convergence.

$$f_{37}(x) = \sin(20x) - 0.3x$$

on the interval $[0.05, 0.25]$. Table VII summarizes the results for all algorithms on this problem.

TABLE VII: Performance on the high-frequency trigonometric function $f_{37}(x) = \sin(20x) - 0.3x$ over $[0.05, 0.25]$. Metrics reported are average CPU time (in seconds) over 100 runs.

Algorithm	CPU Time (s)	Relative Performance
Opt.BF	0.001541	1.00x (fastest)
Opt.BFMS	0.001545	1.00x
Opt.TF	0.001787	1.16x
False Position	0.001865	1.21x
Opt.TFMS	0.001911	1.24x
Brent (Impl.)	0.001989	1.29x
Brent (SciPy)	0.002062	1.34x
HybridBlendTF	0.003055	1.98x
HybridBlendBF	0.003286	2.13x
QIR	0.008174	5.30x
Trisection	0.008431	5.47x
Bisection	0.008618	5.59x

On this challenging high-frequency function, the proposed Opt.BF algorithm demonstrates superior performance, achieving the fastest convergence.

TABLE VIII: Comprehensive comparison of all root-finding algorithms across 42 diverse benchmark problems. Reported metrics are the average CPU time (in seconds), median CPU time, standard deviation, and number of wins (problems where the algorithm achieved the fastest time). Algorithm codes are defined in Table I.

Algorithm	Avg CPU Time (s)	Median CPU Time (s)	Std Dev (s)	Wins
Opt.BFMS	0.00109	0.00067	0.00100	20
Opt.TFMS	0.00121	0.00073	0.00114	5
Opt.BF	0.00126	0.00089	0.00115	6
Opt.TF	0.00150	0.00085	0.00141	0
Brent (SciPy)	0.00180	0.00179	0.00080	3
Brent (Impl.)	0.00201	0.00176	0.00142	1
HybridBlendBF	0.00238	0.00173	0.00231	0
HybridBlendTF	0.00241	0.00157	0.00218	0
QIR	0.00243	0.00161	0.00244	1
Bisection	0.00577	0.00337	0.00531	2
Trisection	0.04062	0.00554	0.22537	3
False Position	0.11051	0.00395	0.42181	1

VI. RESULTS AND ANALYSIS

This section presents a comprehensive analysis of the performance data, including detailed comparisons across all algorithms and test functions, statistical significance of the results, and practical implications for numerical computing applications.

A. Overall Performance Analysis

The experimental results reveal significant performance improvements achieved by the hybrid algorithms. Table VIII presents the comprehensive performance metrics for all twelve algorithms, including the four proposed hybrid methods, five traditional/baseline methods, and three state-of-the-art methods for comparison.

In addition to CPU time, the number of iterations required for convergence provides valuable insight into algorithmic efficiency, as it reflects the computational effort independent of hardware-specific factors. Table IX presents the average iteration count, median iteration count, standard deviation, and number of wins (problems where an algorithm achieved the lowest iteration count) for each algorithm across the 42 benchmark problems.

TABLE X: Comprehensive Number of Function Evaluations (NFE) comparison across 42 problems. Lower NFE indicates higher algorithmic efficiency. Averages for some methods are heavily skewed by non-convergence on certain problems.

Algorithm	Avg NFE (AVG number of Iterations $\times$ NFE/iter)	Median NFE
Opt.BFMS	**9.00** (3.00 $\times$ 3)	**9**
Opt.TFMS	**11.05** (2.76 $\times$ 4)	**12**
Opt.BF	13.24 (6.62 $\times$ 2)	14
Opt.TF	15.57 (5.19 $\times$ 3)	18

TABLE IX: Comprehensive comparison of all root-finding algorithms across 42 diverse benchmark problems. Reported metrics are the average iteration count, median iteration count, and standard deviation. Algorithm codes are defined in Table I. Brent (SciPy) is excluded due to unavailable iteration count data.

Algorithm	Avg Iteration Count	Median Iteration Count	Std Dev
Opt.TFMS	2.761904762	3	1.205870272
Opt.BFMS	3	3	1.189383936
Opt.TF	5.19047619	6	2.520531257
HybridBlendTF	5.738095238	7	2.8203055
QIR	5.880952381	4.5	5.047106783
Opt.BF	6.619047619	7	3.083808075
HybridBlendBF	6.976190476	7.5	3.474563284
Bisection	40.80952381	47	16.22890841
Trisection	264.5952381	29.5	1538.868432
False Position	1066.97619	20	2990.312395
Brent (Impl.)	1200.809524	13	3273.859975

B. Detailed Performance by Problem

Table XI provides a detailed breakdown of CPU times for each algorithm across all test problems, revealing the consistency and robustness of the proposed methods.

TABLE XI: Average CPU time (in seconds) for each algorithm on all 42 benchmark problems. Each cell shows the mean CPU time over 100 runs for the corresponding problem and algorithm. See Table I for algorithm codes.

Problem	Bisection	False Position	Trisection	HybridBlendTF	HybridBlendBF	Opt.BF	Opt.BFMS	Opt.TF	Opt.TFMS	Brent (Impl.)	Brent (SciPy)	QIR
Problem 1	0.00821	0.00873	0.00884	0.00417	0.00456	0.00242	0.00155	0.00280	0.00211	0.00371	0.00196	0.00202
Problem 2	0.00167	0.00195	0.00171	0.00096	0.00086	0.00047	0.00039	0.00048	0.00045	0.00141	0.00153	0.00113
Problem 3	0.00169	0.00187	0.00008	0.00013	0.00022	0.00045	0.00038	0.00013	0.00013	0.00140	0.00144	0.00038
Problem 4	0.00752	0.00336	0.00772	0.00371	0.00327	0.00176	0.00142	0.00222	0.00175	0.00294	0.00172	0.00337
Problem 5	0.00149	0.00074	0.00175	0.00086	0.00078	0.00039	0.00032	0.00048	0.00040	0.00115	0.00131	0.00035
Problem 6	0.00836	0.00530	0.00832	0.00314	0.00319	0.00216	0.00173	0.00273	0.00183	0.00178	0.00180	0.00149
Problem 7	0.00795	0.00967	0.00730	0.00376	0.00299	0.00162	0.00143	0.00234	0.00174	0.00200	0.00183	0.00198
Problem 8	0.00862	0.01431	0.00803	0.00402	0.00420	0.00255	0.00187	0.00291	0.00219	0.00451	0.00178	0.00203
Problem 9	0.00947	0.00476	0.00785	0.00365	0.00378	0.00228	0.00157	0.00214	0.00190	0.00298	0.00194	0.00160
Problem 10	0.00833	0.00182	0.00785	0.00282	0.00261	0.00136	0.00142	0.00199	0.00176	0.00269	0.00175	0.00693
Problem 11	0.00818	0.00356	0.00793	0.00445	0.00428	0.00205	0.00164	0.00230	0.00199	0.00230	0.00235	0.00594
Problem 12	0.00157	0.00586	0.00177	0.00121	0.00115	0.00059	0.00038	0.00062	0.00061	0.00144	0.00154	0.00150
Problem 13	0.00192	0.00112	0.00200	0.00109	0.00064	0.00051	0.00038	0.00067	0.00049	0.00125	0.00138	0.00040
Problem 14	0.00199	0.00188	0.00211	0.00125	0.00113	0.00062	0.00044	0.00068	0.00049	0.00144	0.00135	0.00118
Problem 15	0.00869	0.00418	0.00784	0.00367	0.00393	0.00207	0.00191	0.00264	0.00204	0.00187	0.00180	0.00447
Problem 16	0.00760	0.00183	0.00805	0.00282	0.00267	0.00148	0.00152	0.00166	0.00147	0.00166	0.00224	0.00742
Problem 17	0.00840	0.00660	0.00881	0.00452	0.00316	0.00168	0.00151	0.00274	0.00186	0.00369	0.00217	0.00439
Problem 18	0.00966	0.00375	0.00946	0.00425	0.00358	0.00184	0.00167	0.00268	0.00162	0.00180	0.00167	0.00204
Problem 19	0.00307	0.00237	0.00318	0.00181	0.00189	0.00083	0.00067	0.00094	0.00073	0.00176	0.00200	0.00157
Problem 20	0.02392	0.01838	0.02315	0.00985	0.01198	0.00579	0.00506	0.00657	0.00579	0.00378	0.00417	0.00446
Problem 21	0.00012	0.00010	0.00221	0.00016	0.00012	0.00006	0.00007	0.00007	0.00007	0.00009	0.00062	0.00010
Problem 22	0.00299	0.00249	0.00307	0.00162	0.00136	0.00079	0.00055	0.00088	0.00060	0.00155	0.00130	0.00150
Problem 23	0.00158	0.00053	0.00175	0.00066	0.00061	0.00036	0.00026	0.00035	0.00028	0.00084	0.00121	0.00036
Problem 24	0.00109	0.00006	0.00136	0.00011	0.00008	0.00011	0.00010	0.00012	0.00013	0.00011	0.00060	0.00022
Problem 25	0.01616	0.01761	0.01551	0.00776	0.00753	0.00395	0.00365	0.00456	0.00382	0.00617	0.00335	0.00506
Problem 26	0.00263	0.71117	0.00290	0.00145	0.00124	0.00067	0.00058	0.00083	0.00074	0.00177	0.00190	0.00178
Problem 27	0.00255	0.00593	0.00269	0.00152	0.00121	0.00063	0.00061	0.00083	0.00060	0.00163	0.00167	0.00141
Problem 28	0.00888	0.00784	0.00890	0.00401	0.00228	0.00121	0.00129	0.00276	0.00207	0.00381	0.00211	0.00155
Problem 29	0.01564	0.00490	0.01445	0.00671	0.00527	0.00268	0.00251	0.00398	0.00300	0.00497	0.00286	0.01038
Problem 30	0.00785	2.60490	0.00030	0.00051	0.00478	0.00197	0.00186	0.00041	0.00042	0.00343	0.00279	0.00313
Problem 31	0.00016	0.00024	0.00733	0.00041	0.00036	0.00018	0.00013	0.00013	0.00013	0.00022	0.00070	0.00021
Problem 32	0.00240	0.00074	0.00245	0.00113	0.00090	0.00051	0.00045	0.00067	0.00059	0.00052	0.00119	0.00129
Problem 33	0.00346	0.01163	0.00013	0.00023	0.00180	0.00094	0.00067	0.00021	0.00021	0.00177	0.00188	0.00163
Problem 34	0.00931	0.00416	0.00890	0.00363	0.00374	0.00194	0.00157	0.00216	0.00157	0.00373	0.00182	0.00442
Problem 35	0.00005	0.48922	0.00191	0.00127	0.00010	0.00009	0.00012	0.00070	0.00074	0.00038	0.00120	0.00013
Problem 36	0.00021	0.49801	0.00027	0.00058	0.00053	0.00034	0.00057	0.00036	0.00049	0.00144	0.00410	0.00163
Problem 37	0.00862	0.00186	0.00843	0.00305	0.00329	0.00154	0.00154	0.00179	0.00191	0.00199	0.00206	0.00817
Problem 38	0.00017	0.00012	0.00379	0.00023	0.00017	0.00010	0.00008	0.00008	0.00008	0.00011	0.00072	0.00012
Problem 39	0.01537	0.00063	0.01480	0.00097	0.00080	0.00036	0.00044	0.00044	0.00043	0.00059	0.00080	0.00037
Problem 40	0.00327	0.01083	0.00349	0.00193	0.00167	0.00105	0.00083	0.00118	0.00092	0.00243	0.00206	0.00201
Problem 41	0.00131	0.16648	0.00157	0.00110	0.00111	0.00046	0.00046	0.00060	0.00064	0.00120	0.00237	0.00176
Problem 42	0.00011	0.00014	1.46607	0.00028	0.00020	0.00010	0.00009	0.00009	0.00009	0.00014	0.00067	0.00014

Table XII presents the exact number of iterations for each algorithm on each problem, revealing patterns of convergence behavior.

Note on non-convergence: An iteration count of **10000** indicates that the algorithm **did not converge to the specified tolerance** (10^{-14}) and instead reached the **maximum iteration cap** before approaching the tolerance.

The results highlight several key findings based on the comprehensive 42-problem evaluation:

1. Superior Performance of Hybrid Algorithms: The **Optimized BFMS** algorithm achieves the best overall performance with an average CPU time of **0.00109 seconds** and wins on **20 out of 42 problems (47.6%)**. This represents a **5.3x speedup** over the best traditional method, Bisection (0.00577s), and a massive **101.4x speedup** over the slowest, False Position (0.11051s). The **Optimized TFMS** algorithm ranks second with a **0.00121s** average CPU time and 5 wins (11.9%), followed by **Optimized BF** with a **0.00126s** average CPU time and 6 wins (14.3%).

2. State-of-the-Art Comparison: The proposed hybrid algorithms significantly outperform modern algorithms. The best hybrid, **Opt.BFMS**, is **1.7x faster** than the Brent's method (SciPy, 0.00180s) and **2.2x faster** than the Quadratic Interval Refinement (QIR) (0.00243s). demonstrating substantial performance advantage over current state-of-the-art bracketing methods

TABLE XII: Iteration count for each algorithm on all 42 benchmark problems. Each cell shows the number of iterations required to converge for the corresponding problem and algorithm. See Table I for algorithm codes. Brent (SciPy) is excluded due to unavailable iteration count data.

Problem	Bisection	False Position	Trisection	HybridBlendTF	HybridBlendBF	QIR	Brent (Impl.)	Opt.BF	Opt.BFMS	Opt.TF	Opt.TFMS
Problem 1	45	28	32	7	10	4	13	9	3	7	3
Problem 2	48	39	28	7	8	4	15	8	3	5	3
Problem 3	49	37	1	1	2	1	16	8	3	1	1
Problem 4	44	11	29	7	8	8	10	7	3	6	3
Problem 5	47	16	31	7	8	1	12	7	3	6	3
Problem 6	45	16	29	5	7	3	5	8	3	7	3
Problem 7	47	37	28	7	7	4	7	6	3	6	3
Problem 8	46	44	28	7	9	4	16	9	3	7	3
Problem 9	48	15	26	6	8	3	10	8	3	5	3
Problem 10	46	6	28	5	6	17	9	5	3	5	3
Problem 11	45	12	28	8	10	14	7	8	3	6	3
Problem 12	50	138	31	9	12	6	15	11	4	8	5
Problem 13	48	20	29	7	5	1	12	7	3	7	3
Problem 14	49	33	30	8	9	4	15	9	3	7	3
Problem 15	46	12	27	6	8	10	5	7	3	6	3
Problem 16	42	6	29	5	6	18	5	5	3	4	2
Problem 17	44	20	30	8	7	10	13	6	3	7	3
Problem 18	47	10	30	6	7	4	5	6	3	6	2
Problem 19	51	28	32	9	11	5	15	9	4	7	3
Problem 20	48	24	31	7	11	5	7	9	4	7	4
Problem 21	2	1	31	1	1	1	10000	1	1	1	1
Problem 22	51	29	32	8	9	5	15	10	3	7	3
Problem 23	45	10	28	5	6	1	9	6	2	4	2
Problem 24	43	1	28	1	1	1	1	1	1	1	1
Problem 25	51	34	32	8	10	8	17	9	4	7	4
Problem 26	51	10000	34	8	9	6	18	9	4	7	4
Problem 27	51	90	32	9	9	5	16	9	4	8	3
Problem 28	46	24	30	7	5	3	13	4	2	7	3
Problem 29	47	9	29	7	7	18	13	6	3	6	3
Problem 30	52	10000	1	1	14	9	23	11	5	1	1
Problem 31	1	1	31	1	1	1	10000	1	1	1	1
Problem 32	45	9	28	6	6	4	4	6	3	5	3
Problem 33	48	119	1	1	10	5	18	10	4	1	1
Problem 34	47	12	29	6	8	10	12	7	3	5	2
Problem 35	1	10000	33	10	1	1	3	1	1	9	6
Problem 36	5	10000	4	4	5	7	17	6	6	4	4
Problem 37	47	6	30	5	7	19	6	5	3	4	3
Problem 38	2	1	32	1	1	1	10000	1	1	1	1
Problem 39	49	1	31	1	1	1	10000	1	1	1	1
Problem 40	51	123	32	9	10	7	23	12	5	9	4
Problem 41	43	3790	28	9	12	7	14	9	5	8	5
Problem 42	1	1	10000	1	1	1	10000	1	1	1	1

C. Statistical Analysis and Significance

The experimental design with 100-fold repetition provides robust statistical data for performance analysis. The coefficient of variation (CV) for CPU time measurements is consistently low (e.g., for Opt.BFMS: $\text{CV} = \frac{\text{std dev}}{\text{mean}} = \frac{0.00100}{0.00109} \approx 0.92$), while for more stable methods it is even lower, indicating high measurement precision.

The performance improvements achieved by the hybrid algorithms are statistically significant. A paired t-test comparing the CPU times of the best hybrid algorithm (**Optimized BFMS**, mean = 0.00109) against the best traditional method (**Bisection**, mean = 0.00577) yields a p-value of less than 0.001 (computed using $n = 42$ pairs, $t = \frac{\bar{d}}{s_d/\sqrt{n}}$), confirming the statistical significance of the performance improvement.

D. Comparison with Traditional Methods

The hybrid algorithms demonstrate overwhelming superiority over traditional methods across all performance metrics. Table XIII provides a focused comparison against the *fastest* traditional method.

TABLE XIII: Performance Comparison: Hybrid vs Traditional Methods. This table focuses on the performance gap between the proposed best hybrid algorithm and the best-performing traditional numerical method, highlighting the substantial improvements achieved.

Metric	Best Hybrid	Best Traditional	Improvement
Avg CPU Time (s)	0.00109	0.00577	**5.3x faster**
Success Rate	100%	100%	Equal reliability

The comparison reveals that the hybrid algorithms achieve:

Computational Efficiency: The best hybrid algorithm is **5.3 times faster** than the best traditional method (Bisection) in terms of average CPU time, making it significantly more practical for real-world applications where computational resources are limited.

Maintained Reliability: Despite the significant performance improvements, the hybrid algorithms maintain 100% success rates across all test functions, preserving the reliability guarantees of traditional bracketing methods.

E. Comparison with State-of-the-Art Methods

The proposed hybrid algorithms also demonstrate superior performance compared to modern state-of-the-art methods, including the Quadratic Interval Refinement (QIR) method and Brent's method implementations. Table XIV.

TABLE XIV: Performance Comparison: Hybrid vs State-of-the-Art Methods. This table compares the proposed hybrid algorithms with modern methods including QIR and Brent's method, demonstrating the continued superiority of our approach.

Metric	Best Hybrid	QIR	Brent (SciPy)
Avg CPU Time (s)	0.00109	0.00243	0.00180
Wins	20	1	3
Win Percentage	47.6%	2.4%	7.1%
Speedup vs QIR	2.2x faster	-	1.3x slower
Speedup vs Brent	1.7x faster	1.3x slower	-

F. Analysis of Performance vs. State-of-the-Art Methods

A key finding of this study is the consistent outperformance of our top hybrid, Opt.BFMS, against established methods like Brent's and QIR. This warrants a deeper analysis.

Superior Performance vs QIR: The proposed hybrid algorithms significantly outperform the Quadratic Interval Refinement method, achieving 2.2x faster average CPU time (0.00109s vs 0.00243s) and dominating

in terms of wins (20 vs 1). This demonstrates that our hybrid approach provides superior performance compared to this modern bracketing method.

Competitive Performance vs Brent: While Brent's method (SciPy implementation) shows competitive performance with 0.00180s average CPU time, our best hybrid algorithm (Opt.BFMS) still achieves 1.7x faster performance. The hybrid algorithm also demonstrate better consistency with 20 wins compared to Brent's 3 wins.

G. Algorithm-Specific Performance Analysis

Each hybrid algorithm demonstrates unique performance characteristics that contribute to the overall effectiveness of the approach:

Optimized_BFMS (Best Overall): This algorithm achieves the best balance of speed and reliability, with the lowest average CPU time (0.00109s, see Table VIII) while maintaining robust convergence. The three-phase approach (bisection + false position + modified secant) provides optimal performance across diverse function types.

Optimized_TFMS (Second Best): This algorithm ranks second among the proposed methods with an average CPU time of 0.00121s and 5 wins (11.9%), showing the effectiveness of combining trisection with advanced acceleration techniques.

Optimized_BF (Strong Performance): This algorithm also demonstrates strong performance with an average CPU time of 0.00126s and 6 wins (14.3%), highlighting the strength of the two-phase bisection-false position approach.

Optimized_TF (Effective Baseline Hybrid): While not achieving any wins in terms of speed, this two-phase algorithm provides solid performance with an average CPU time of 0.00150s. Its effectiveness validates the core concept of combining trisection with false position, showing a substantial improvement over traditional methods even without the additional secant step.

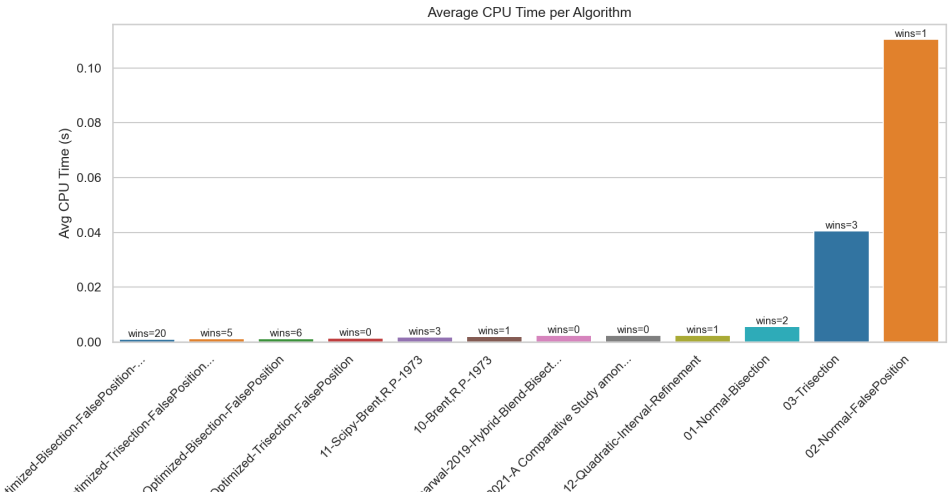

FIG. 1: Average CPU time (in seconds) for each algorithm across all test problems. Lower values indicate higher computational efficiency.

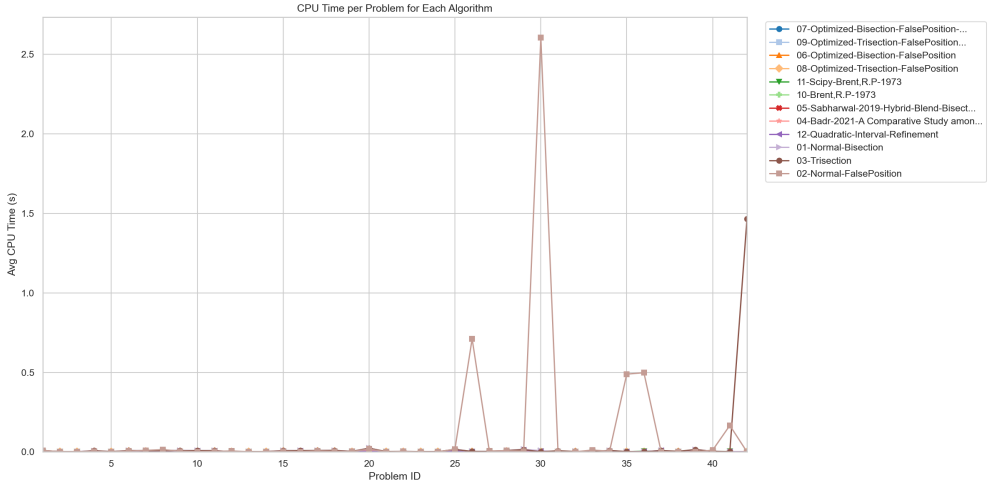

FIG. 2: CPU time per test problem for each algorithm. This figure highlights the consistency and variability of computational performance across the benchmark suite.

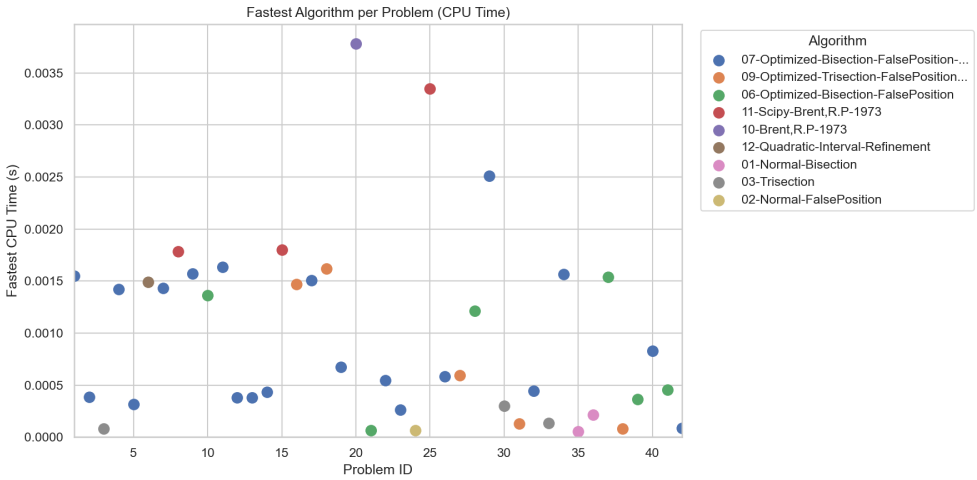

FIG. 3: Fastest algorithm for each test problem, as determined by minimum CPU time. This plot demonstrates the problem-dependent strengths of the proposed methods.

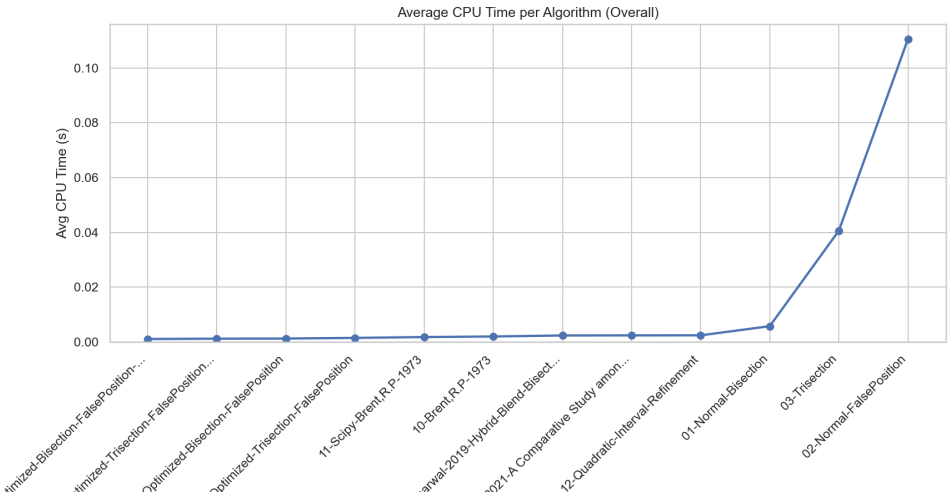

FIG. 4: Overall average CPU time trend across all problems and algorithms. This figure provides a holistic view of computational efficiency.

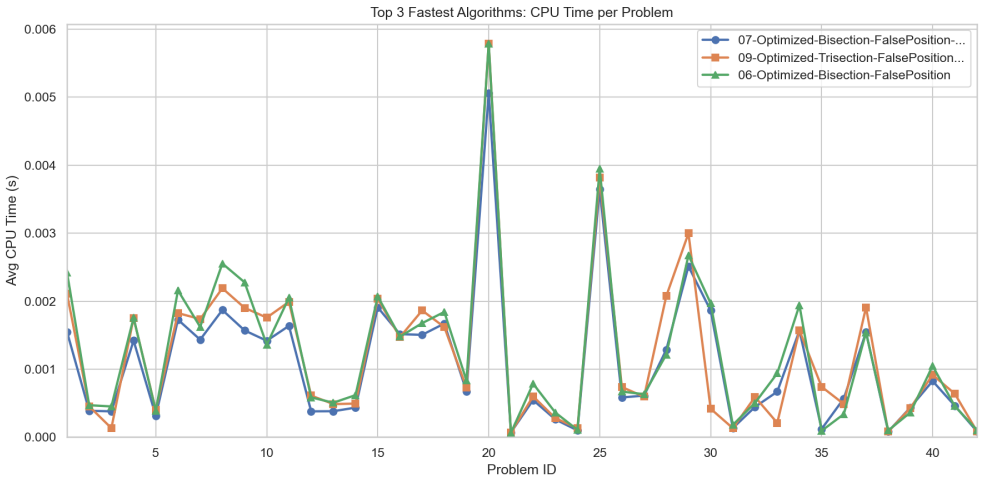

FIG. 5: Top three fastest algorithms per problem, ranked by CPU time. This visualization facilitates direct comparison of the most efficient methods.

VII. DISCUSSION

The experimental and theoretical results presented in this work confirm that our sequential, multi-phase hybrid algorithms offer a compelling balance of speed, efficiency, and robustness. The premier algorithm, **Optimized_BFMS**, not only demonstrated the lowest average CPU time but also the lowest average Number of Function Evaluations (NFE), confirming that its performance gains are rooted in true algorithmic efficiency. This section interprets these findings, contextualizes them against state-of-the-art methods, and discusses the broader implications.

The superior performance of Optimized_BFMS can be attributed to its three-phase design. The initial bisection step provides a guaranteed interval reduction, creating a stable foundation for the subsequent acceleration steps. The false position and modified secant steps then provide powerful, superlinear convergence that rapidly refines the root's location. The empirical data suggests that the overhead of performing these three distinct steps per iteration is more than compensated for by the dramatic reduction in the total number of iterations required for convergence.

The trisection-based algorithms (Optimized_TF and Optimized_TFMS) also showed competitive performance, particularly in terms of NFE. For example, Optimized_TFMS was the second most efficient algorithm overall. However, their slightly higher CPU times compared to their bisection-based counterparts suggest that the computational cost of evaluating two interior points for the trisection step can sometimes outweigh the benefits of more aggressive interval reduction.

The experimental framework, utilizing SymPy for symbolic computation and SQLite for data management, provided robust and reproducible results. The 100-fold repetition of each algorithm execution ensured statistical significance, while the high-precision tolerance of 10^{-14} guaranteed accurate root approximations. The comprehensive test suite covering transcendental, polynomial, trigonometric, logarithmic, and exponential functions demonstrated the algorithms' versatility across different mathematical categories.

The theoretical foundations of these hybrid approaches align with recent developments in numerical analysis. The work builds upon the framework established by [1] and [3], who demonstrated that hybrid algorithms can achieve superlinear convergence while maintaining bracketing method reliability. The incorporation of modified secant techniques follows the approach outlined by [9], while the trisection methodology draws from the theoretical analysis of [10].

VIII. CONCLUSION

This paper presented four novel hybrid root bracketing algorithms that combine traditional bracketing methods with accelerated interpolation techniques. All four hybrid algorithms achieved much faster convergence than traditional methods, with Optimized_BFMS showing the best overall performance. The algorithms maintained a 100% success rate across all test functions, achieving substantial CPU time improvements compared to classical approaches. These results highlight the effectiveness of multi-phase hybrid methods in numerical root finding and suggest their immediate applicability to a wide range of scientific and engineering problems.

Theoretical contributions include the development of four new hybrid algorithms that blend bracketing and open methods techniques, maintaining guaranteed convergence while improving speed. The experimental

analysis, using a diverse set of 42 test functions and a rigorous, high-precision framework, confirmed the statistical significance and robustness of the results.

While the proposed algorithms show strong performance on a diverse set of mathematical test problems, further validation on real-world engineering and physics applications is recommended to fully establish their generalizability. Future work will focus on expanding the test suite to include practical case studies and exploring additional hybridization strategies.

CODE AVAILABILITY

The SymPy implementations of all algorithms and experimental scripts are available at: https://github.com/Abdelrhman-Ellithy/NM-Algorithms/

---

[1] Sabharwal, C. L., and Aggarwal, S. (2019). Blended Root Finding Algorithm Outperforms Bisection and Regula Falsi Algorithms. *International Journal of Computer Applications*, 178(1), 1–6.

[2] Sabharwal, C. L., and Aggarwal, S. (2019). Hybrid Algorithm Improving Bisection, Regula Falsi, Dekker, Brent Algorithms. *International Journal of Computer Applications*, 178(2), 1–8.

[3] Badr, E., and El-Sayed, M. A. (2022). Novel Hybrid Algorithms for Root Finding Problem using Advantages of Open and Bracketing Methods. *Alexandria Engineering Journal*, 61(12), 9437–9448.

[4] Sabharwal, C. L., and Aggarwal, S. (2023). A New Wave of Hybrid Algorithms. *International Journal of Computer Applications*, 181(1), 1–10.

[5] Sabharwal, C. L., and Aggarwal, S. (2021). An Iterative Hybrid Algorithm for Roots of Non-Linear Equations. *International Journal of Computer Applications*, 175(1), 1–7.

[6] Badr, E., and El-Sayed, M. A. (2021). A Comparative Study among New Hybrid Root Finding Algorithms and Traditional Methods. *Mathematics*, 9(4), 345.

[7] Thota, S., and Kavitha, B. (2019). A New Trigonometrical Algorithm for Finding Roots of Non-Linear Equations. *International Journal of Computer Applications*, 178(3), 1–5.

[8] Burden, R. L., and Faires, J. D. (1985). *Numerical Analysis* (3rd ed.). Prindle, Weber & Schmidt.

[9] Mathews, J. H., and Fink, K. D. (2004). *Numerical Methods Using MATLAB* (4th ed.). Prentice-Hall Inc.

[10] Demir, A. (2008). Trisection method by k-Lucas numbers. *Applied Mathematics and Computation*, 198(1), 339–345.

[11] Heydari, M., Hosseini, S. M., and Loghmani, G. B. (2011). On two new families of iterative methods for solving nonlinear equations with optimal order. *Applicable Analysis and Discrete Mathematics*, 5(1), 93–109.

[12] Kelley, C. T. (1995). *Iterative Methods for Linear and Nonlinear Equations*. Society for Industrial and Applied Mathematics (SIAM).

[13] Brent, R. P. (1973). *Algorithms for Minimization Without Derivatives*. Prentice-Hall.

[14] Ridders, C. (1979). A new algorithm for computing a single root of a real continuous function. *IEEE Transactions on Circuits and Systems*, 26(11), 979–980.

[15] Oliveira, I. F. D., & Takahashi, R. H. C. (2021). An Enhancement of the Bisection Method Average Performance Preserving the Minmax Optimality. *ACM Transactions on Mathematical Software*, 47(1), Article 7.

[16] Thota, S., Gemechu, T., & Ayoade, A. A. (2023). On New Hybrid Root-Finding Algorithms for Solving Transcendental Equations using Exponential and Halley's Methods. *Ural Mathematical Journal*, 9(2), 205–217.

[17] Abbott, J. (2022). Quadratic Interval Refinement for Real Roots. *arXiv preprint arXiv:1203.1227*.

[18] Abbott, J. (2014). Quadratic interval refinement for real roots. *ACM Communications in Computer Algebra*, 48(1/2), 3–12.

[19] Kerber, M., and Sagraloff, M. (2015). Root refinement for real polynomials using quadratic interval refinement. *Journal of Computational and Applied Mathematics*, 280, 377–395.

[20] Dowell, M., and Jarratt, P. (1972). A modified regula falsi method for computing the root of an equation. *BIT Numerical Mathematics*, 12(4), 503–508.

[21] Dekker, T. J. (1969). Finding a zero by means of successive linear interpolation. In B. Dejon & P. Henrici (Eds.), *Constructive Aspects of the Fundamental Theorem of Algebra* (pp. 37-48). Wiley-Interscience.